

Rising Beyond Borders

Document 21 — Contact Forms, Email & Submission Integration

Master Specification + Claude Code Implementation Prompt

Purpose: turn the existing disabled Contact, Volunteer, Partner, Fundraise and Newsletter forms into controlled production submission flows with a server-side boundary and Resend email delivery, while preserving the existing static/pre-rendered frontend architecture.

Security note: the API credential supplied in the conversation is treated as a secret and is intentionally **not reproduced in this document**. Because it was pasted into chat, rotate/revoke that key and create a replacement before production use. The replacement must be stored only as a server-side environment secret.

Resend supports server-side email sending through its API/SDK and recommends keeping the API key in an environment variable rather than browser code. Use a sending-scoped/domain-restricted key where practical. ■cite■turn0search1■turn0search2■

1. Scope

- Enable the existing Contact form.
- Enable the existing Volunteer form.
- Enable the existing Partner form.
- Enable the existing Fundraise form.
- Define the Newsletter signup flow.
- Add a server-side submission endpoint/function boundary.
- Validate and normalize all incoming data server-side.
- Add anti-spam and rate-limiting integration points.
- Send structured notification emails through Resend.
- Send structured acknowledgement emails where approved and appropriate.
- Create reusable, responsive, branded email templates.
- Keep recipient addresses and provider credentials out of browser code.
- Update CSP, technology audit, privacy dependencies and deployment documentation.
- Provide test/staging behavior without sending accidental production emails.

2. Architecture Decision

The current site is a static/pre-rendered React application. Resend's API key must not be exposed to the browser. Therefore forms must submit to a server-side function/API route supplied by the eventual hosting platform or a separately approved serverless runtime. Do not turn the frontend into a traditional server application solely for email.

- Browser → same-origin submission endpoint → server-side validation → anti-spam/rate-limit checks → Resend → recipient inbox.
- Optional acknowledgement: endpoint → Resend → submitter email.
- The browser never calls `api.resend.com` directly with the API key.
- The API key never uses a VITE_* variable because Vite browser variables are public.
- The endpoint should return a generic success/failure result without exposing Resend internals.

3. Environment & Secret Configuration

The requested environment variable name for the server-side secret is **EMAIL_API_KEY**. It must exist only in the server/runtime environment. If the selected deployment platform has a convention requiring ``RESEND_API_KEY``, map the platform secret into the application's server-only configuration without exposing either variable to client bundles.

- Never place EMAIL_API_KEY in src/, VITE_* variables, public/, HTML, content files or client JavaScript.
- Never log EMAIL_API_KEY.
- Never echo environment variables from an API response.
- Use a new rotated key because the credential supplied during planning is now exposed in conversation context.
- Prefer a Resend sending-access key restricted to the verified sending domain where the Resend account supports that configuration.
- Document the variable name and required deployment configuration, not its value.

4. Form Inventory

Form	Purpose	Fields	Primary email
Contact	General inquiry	Name, email, optional topic, message	RBB-configured recipient
Volunteer	Volunteer interest	Name, email, message/interest fields approved by RBB	RBB-configured recipient
Partner	Partnership inquiry	Name, email, organization, message	RBB-configured recipient
Fundraise	Fundraising inquiry	Name, email, message/plan details approved by RBB	RBB-configured recipient
Newsletter	Newsletter subscription	Email + approved consent	Newsletter service/recipient architecture pending app

Do not expand fields simply because the endpoint could accept them. Every field must have an approved business purpose, privacy basis and email-template representation.

5. Server-Side Validation

- Validate HTTP method and Content-Type.
- Parse JSON safely and reject malformed bodies.
- Enforce maximum request body size.
- Trim appropriate text fields.
- Validate email syntax server-side.
- Enforce field-specific maximum lengths.
- Reject unexpected fields or explicitly strip them before processing.
- Normalize line endings and protect email-template rendering from HTML injection.
- Escape user content before inserting it into HTML email.
- Do not trust client-side validation.
- Return field-level validation errors only where safe and useful.
- Do not reveal whether an internal recipient exists.

6. Anti-Spam & Rate Limiting

- Implement a server-side rate-limit boundary per IP and form/action.
- Use a honeypot or equivalent low-friction control as a baseline.
- Allow a stronger challenge provider to be added later without changing form UX architecture.
- Reject obvious automated submissions such as filled honeypot fields or impossible timing where appropriate.
- Do not rely on client-side hidden fields as a security boundary.
- Avoid storing IP addresses in application logs unless the selected security/privacy design explicitly requires it.
- Document the chosen retention/logging behavior before launch.

7. Resend Email Architecture

- Use the official Resend Node SDK or direct REST API from the server-side runtime.
- Use a verified RBB sending domain before production delivery.
- Use a stable From address approved by RBB.
- Use Reply-To to route a submission reply to the submitter where appropriate; do not put the submitter in From unless explicitly designed and authorized.
- Recipient addresses must come from server-side approved configuration, not browser input.
- Use idempotency keys where retry behavior could create duplicate sends; Resend documents idempotency support for email requests. ■cite■turn0search1■
- Capture the Resend message ID server-side for safe operational diagnostics without logging message content unnecessarily.
- Do not expose provider error payloads to users.

8. Email Template System

Every form must use a dedicated, well-structured email template. Do not send raw concatenated text or unstyled HTML from the endpoint.

Template	Recipient	Required structure
Contact notification	RBB inbox	Brand header → inquiry type → sender details → message → submission timestamp/reference → reply action
Contact acknowledgement	Submitter	Thank-you header → received summary → what happens next wording approved by RBB → contact fallback
Volunteer notification	RBB inbox	Volunteer interest → applicant details → message → reference → reply action
Volunteer acknowledgement	Submitter	Thank you → interest received → next-step wording only if approved
Partner notification	RBB inbox	Partnership inquiry → organization/contact details → message → reference
Partner acknowledgement	Submitter	Thank you → inquiry received → approved next-step wording
Fundraise notification	RBB inbox	Fundraising inquiry → contact/details → plan/message → reference
Fundraise acknowledgement	Submitter	Thank you → inquiry received → approved next-step wording
Newsletter confirmation	Subscriber	Brand header → subscription confirmation → approved consent/unsubscribe language

9. Email Design System

- Use RBB brand colors: Deep Trust Blue #10437C, Bright Sky Blue #00ADEF, Light Gray #F5F7FA, White #FFFFFF, Charcoal #3A3A3A, Growth Green #5CB85C.
- Use a resilient email-safe layout rather than assuming modern browser CSS.

- Include plain-text fallback for important transactional messages.
- Use accessible contrast and readable typography.
- Use the RBB logo only after confirming the approved production logo asset and its public URL.
- Do not use unapproved photography in email headers.
- Keep buttons and links clear and keyboard/screen-reader understandable.
- Include organization identity and appropriate footer information once RBB supplies the final approved details.
- Do not add marketing language or claims that are not approved.

10. User Experience States

- Idle: normal form.
- Client validation error: clear field-level messages and summary.
- Submitting: prevent duplicate submissions and show progress.
- Success: clear confirmation without claiming a response time unless RBB approves one.
- Server validation error: actionable message.
- Rate limited: ask user to wait/retry later without revealing internal thresholds.
- Provider failure: generic message; log operational details server-side.
- Network failure: generic retry message.
- Spam rejection: do not reveal anti-spam implementation details.

11. Privacy & Consent

- Forms must remain disabled unless a published Privacy Policy exists, as established in Document 13.
- The form should link to the canonical Privacy Policy when published.
- Collect only fields needed for the stated purpose.
- Do not add marketing/newsletter consent to general forms unless explicitly designed and approved.
- Newsletter subscription requires explicit consent language, unsubscribe handling and a defined subscriber-management process.
- Do not promise data deletion, retention periods or response times until RBB approves the policy.
- Do not place submitted personal data in client-side analytics events.

12. Submission Storage

Document 21 does not require building a database. The initial implementation may be email-forwarding only, provided RBB accepts that operational model. If persistent submission storage is later required, it must be a separate approved architecture decision with retention, access control, backups and privacy rules.

- Do not silently create a database just to make forms work.
- Do not write submissions to local filesystem storage in a serverless deployment.
- Do not return submitted data in a public API response.
- Do not log complete submissions by default.

13. Resend Domain & DNS Readiness

- RBB must provide/approve the sending domain and From address.
- The domain must be verified in Resend before production sending.
- DNS records required by Resend must be configured at the domain host.
- Do not use `onboarding@resend.dev` or another temporary sender for production.
- Test delivery to approved addresses before enabling public submission.
- Check SPF/DKIM/domain status and record the verification state in deployment documentation.
- A verified sending domain is a production prerequisite.

14. CSP & Technology Audit Changes

- The submission endpoint itself may require no new browser origin if it is same-origin.
- Resend is server-to-server; the browser CSP should not need direct `api.resend.com` access.
- If a CAPTCHA or external client-side anti-spam provider is approved, its origins must be explicitly added to the CSP and technology audit.
- Email assets hosted externally should be avoided unless required; prefer stable public RBB assets.
- Re-run the technology audit after integration to detect cookies, storage or new scripts.

15. Error Handling & Observability

- Generate a non-sensitive request/reference ID for operational troubleshooting.
- Log only necessary metadata: form type, timestamp, success/failure category and provider message ID where useful.

- Do not log message bodies, API keys, passwords, payment data or full personal details.
- Do not expose stack traces or Resend API responses to the browser.
- Provider failures should be distinguishable internally from validation/spam failures.
- If the existing no-op monitoring layer is enabled later, ensure its event schema remains privacy-safe.

16. Test Matrix

Test	Expected result
Valid Contact	Server accepts and sends approved notification template.
Valid Volunteer/Partner/Fundraise	Correct template and routing used.
Newsletter	Only enabled after consent/provider requirements are satisfied.
Invalid email	Rejected server-side.
Oversized body	Rejected without processing.
Unknown field	Rejected or safely ignored according to documented policy.
HTML/script payload	Escaped; never executable in email or page.
Header injection payload	Rejected/normalized; cannot manipulate email headers.
Honeypot	Rejected without sending.
Rate limit	Rejected with generic response.
Resend failure	Generic user error; internal diagnostic retained.
Duplicate retry	Idempotency/retry behavior prevents unintended duplicate sends where configured.
API key scan	Key absent from client bundle and repository.
JS-off	Form behavior remains understandable; server endpoint still protected.
Accessibility	No new axe violations; keyboard/error/success states work.
CSP	No new browser-origin violation unless explicitly approved.

17. Claude Code Implementation Prompt

You are implementing Document 21 – Contact Forms, Email & Submission Integration for the Rising Beyond Borders website.

GOAL

Enable the existing Contact, Volunteer, Partner, Fundraise and Newsletter flows using a secure server-side submission boundary and Resend email delivery.

IMPORTANT SECRET RULE

A Resend API credential was supplied during planning. Do NOT copy it into source code, documentation, the PDF, tests, chat output, or any VITE_* variable. Treat it as compromised because it was exposed during planning. The project owner must rotate/revoke it and create a replacement before production use.

SERVER SECRET

Use the server-only environment variable:

EMAIL_API_KEY

Do not expose it to the browser. If the deployment platform requires RESEND_API_KEY, map it server-side without exposing it client-side.

ARCHITECTURE

Browser -> same-origin serverless/API endpoint -> validation -> anti-spam/rate-limit -> Resend -> approved RBB recipient.

Optional acknowledgement -> Resend -> submitter.

Do not call api.resend.com from browser code.

IMPORTANT

- Do not add a traditional backend/server application if the selected host supports serverless functions.
- Do not add a database in this phase.
- Do not invent RBB recipient email addresses.
- Do not invent From addresses.
- Do not enable production forms until Privacy Policy is published and RBB has approved recipients/sender domain.
- Do not use onboarding@resend.dev for production.
- Do not stage or commit.

FORMS

1. Contact: name, email, optional topic, message.
2. Volunteer: use currently approved fields only.
3. Partner: name, email, organization, message.
4. Fundraise: name, email, approved fundraising details/message.
5. Newsletter: email + explicit approved consent.

VALIDATION

Implement server-side validation for method, content type, JSON, size, fields, lengths, email, unexpected fields, header-injection patterns and HTML/script payloads. Escape all user content before HTML email rendering.

ANTI-SPAM

Implement a server-side rate-limit boundary and honeypot. Keep the architecture ready for a stronger challenge provider without requiring it now.

RESEND

Use the official Resend Node SDK or REST API server-side.

Use EMAIL_API_KEY only on the server.

Use an approved verified sending domain and approved From address.

Use Reply-To for the submitter when appropriate.

Use server-side approved recipient configuration.

Use idempotency keys for retry-sensitive sends.

Do not expose provider errors to users.

EMAIL TEMPLATES

Create reusable branded templates for:

- contact notification
- contact acknowledgement
- volunteer notification
- volunteer acknowledgement
- partner notification
- partner acknowledgement
- fundraise notification
- fundraise acknowledgement
- newsletter confirmation

Templates must be responsive, accessible, email-client resilient and use RBB brand colors. Do not invent organization details or response-time promises.

PRIVACY

Require published Privacy Policy before public forms activate. Do not collect unnecessary fields. Newsletter requires explicit consent, unsubscribe handling and an approved subscriber-management process.

CONFIGURATION

Keep recipient addresses, From addresses, provider settings and server secrets out of browser content. Use approved server-side configuration.

TESTING

Test valid/invalid submissions, oversized bodies, unknown fields, HTML/script injection, email-header injection, honeypot, rate limiting, Resend failures, retries/idempotency, accessibility, CSP, JS-off behavior and client-bundle secret scanning.

DOCUMENTATION

Update deployment, technology-audit, security and privacy-related documentation with the actual architecture and unresolved RBB decisions.

REPORT

Return:

- files changed
- endpoint/function architecture
- forms enabled/disabled
- Resend integration status
- template list
- environment variables used (names only, never values)
- sender/recipient configuration status
- anti-spam/rate-limit implementation
- privacy/consent state
- tests and results
- CSP/origin changes
- technology-audit result
- unresolved RBB decisions
- confirmation that no secret appears in source/client bundle
- whether anything was staged or committed

Do not claim delivery, compliance or security success beyond what was actually tested.

18. RBB Decisions Required Before Public Activation

- Production sending domain.
- Approved From address.
- Approved recipient for Contact.

- Approved recipient for Volunteer.
- Approved recipient for Partner.
- Approved recipient for Fundraise.
- Newsletter service/recipient architecture.
- Whether submitter acknowledgement emails should be enabled for each form.
- Final form fields.
- Privacy Policy publication.
- Consent wording and newsletter unsubscribe process.
- Anti-spam provider, if a stronger challenge is desired.
- Rate-limit thresholds/operational policy.
- Email footer organization details.
- Final email copy and response-time wording, if any.

19. Acceptance Criteria

- No Resend API key is present in client code, source control or generated browser bundles.
- Forms submit through a server-side boundary.
- All incoming data is validated server-side.
- Anti-spam and rate limiting exist before public activation.
- Every enabled form has a structured email template.
- Recipient and sender configuration is server-side and approved.
- Resend domain verification is complete before production sending.
- Privacy Policy and consent dependencies are satisfied.
- CSP and technology audit remain clean.
- No database or unnecessary backend is introduced.
- Accessible loading, validation, success and failure states work.
- Provider failures do not leak internal details.
- No secrets appear in logs or responses.
- Production and test sending can be clearly separated.

Document status: Implementation specification. Resend is the approved email provider for this phase; production activation remains dependent on domain, sender, recipient, privacy and deployment decisions.